START SMALL · UNDERSTAND THE MECHANISM

AI Engineering From First Principles

30 daily lessons

Concept → rule and derivation → worked example → practice → explain it.

Revised October 15, 2026

Read at your own pace. Each lesson defines its symbols and separates the general rule from the numbers used to illustrate it.

Your learning route

1. A small app, a clear goal, and a new way to learn
2. Numbers, vectors and matrices without the mystery
3. Scores become probabilities: softmax and temperature
4. Build your first language predictor with counts
5. Turn text into tokens, then into embeddings
6. Loss and gradients: teach one number to improve
7. The training loop, batches and honest evaluation
8. Neural networks and autograd, one layer at a time
9. Attention: let one word consult the others
10. Assemble a tiny transformer and follow one token
11. Position, context limits and the KV cache
12. Choose output tokens and understand inference optimizations
13. Prompting, context and JSON that deserves checking
14. Embeddings and search: find meaning without exact words
15. Build RAG from a question and two documents
16. Hybrid ranking and knowledge graphs without the hype
17. Tools and human review: from a suggestion to an action
18. Agents, memory and surviving a crash
19. Safety and prompt injection in plain language
20. MCP and A2A: understand the interface before the acronym
21. Evaluation and statistics you can actually use
22. Serving, latency, cost and model routing
23. LLMOps, architecture and data engineering
24. Fine-tuning, LoRA and QLoRA through your own adapter
25. Preferences, reinforcement learning and distillation
26. Multimodal AI and the complete NoteEchoes journey
27. Coding, SQL, system design and behavioral interviews
28. Your capstone and a learning plan that allows understanding
29. Your parameter map and a gentle glossary
30. Optional workshop: read and run the complete tiny decoder

1. A small app, a clear goal, and a new way to learn

The concept: what makes a model different from an application?

A **model** is a calculation whose adjustable numbers have been fitted to data. An **application** is the surrounding program that accepts requests, stores information, checks permissions and performs operations. Understanding that boundary comes before choosing a model.

A calculation has a **structure** and **values inside that structure**. We choose the structure: which operations to perform and how to connect them. Training changes the adjustable values, called **weights** or **parameters**, so that the outputs better match the training targets. Inference means computing an output using the current weights. An ordinary inference request does not train the model.

The rule and what its symbols mean

We can describe a model generally as:

$$\hat{y} = f_\theta(x)$$

Read this as “the predicted output is a function of the input, using parameters theta.” x is input data; $\hat{y}$ means a prediction, with the hat distinguishing it from the correct target y. f names the calculation; θ is a compact name for all its learned numbers. The subscript does not mean multiplication.

This notation describes a family of models, not one universal prediction formula. A particular model might multiply numbers; another might transform a sequence of token vectors through many layers. Training chooses parameter values within the structure we supplied.

Why the application needs more than a prediction

A model outputs a representation: perhaps scores, text or proposed fields. An external action requires a separate program to interpret and validate those fields, establish authority and call the appropriate service. Therefore the operational flow is **input** → **prediction** → **validation** → **authorized execution** → **recorded outcome**. The prediction alone cannot establish that an action happened.

A **hyperparameter** is a setting for the learning procedure, such as learning rate. A **runtime setting** controls inference, such as output length. A **metric** measures behavior. These play different roles even though all may be stored as numbers in configuration files.

Worked example: a rule before a learned model

The lab below is deliberately a hand-written rule, with no learned weights. It illustrates the interpretation boundary before we replace rules with a learned calculation.

```
text = "Remind me to call Maya tomorrow at 6 pm"
if "remind" in text.lower():
    proposal = {"intent": "reminder", "needs_review": True}
else:
    proposal = {"intent": "note", "needs_review": False}
print(proposal)
```

The condition checks whether the lowercase text contains `remind`. It returns a dictionary describing a proposal. `needs_review` is a field in that dictionary; it does not itself open a review dialog or schedule anything. Application code must implement those behaviors.

Changing the text to “Don't remind me” exposes the rule's limitation: it detects the same substring but misses the negation. A learned model may handle that pattern better, but we must measure whether it does.

Check your understanding

Question: Who decides the calculation, and who determines whether a reminder was actually created?

Answer: We choose the model structure; training adjusts its weights. The application validates and executes a proposal and records the service's result. A generated sentence claiming success is insufficient evidence of execution.

Connect it to your project

For NoteEchoes, trace audio capture, transcription, intent prediction, validation and execution separately. Compare the artifacts at each boundary to locate the first error. Define a few correct outcomes, including clarification and cancellation, before evaluating a more complicated model. Complexity is useful only when it improves the behavior you need.

2. Numbers, vectors and matrices without the mystery

The concept: a model combines numerical features

A **scalar** is one number. A **vector** is an ordered list of numbers. A **matrix** is a rectangular table. A **tensor** is an array with any number of axes. These names describe how information is arranged; they do not specify what the information means.

To produce one output from several input features, we need a rule for combining their contributions. A **linear layer** gives each input a weight, adds the weighted contributions and adds an optional offset called a bias. The technical term for the rule with a bias is affine.

The formula and every symbol

For one output:

$$\hat{y} = \sum_{i=1}^{D} w_i x_i + b$$

x_i is input feature i, w_i is its weight, D is the number of input features, and b is the bias. $\sum$ means "add the following expression over all the indicated indices." The prediction is $\hat{y}$. With two inputs, this expands to `w1*x1 + w2*x2 + b`.

Why multiply and add? Multiplication scales an input's contribution; a negative weight reverses its sign. Addition combines the contributions. Bias allows a nonzero output even when every input is zero. This is a chosen model family, not a law saying every relationship must be linear.

Derive the matrix form

If we need several outputs, give each output its own weight vector and bias. Stack those vectors as **rows** of a matrix W:

$$\hat{y}_j = \sum_{i=1}^{D} W_{ji} x_i + b_j, \qquad \hat{y} = Wx + b$$

Here j labels an output and i labels an input. With O outputs, W has shape `[O, D]`, x has length `D`, and the result has length `O`. Each row consumes the same `D` inputs and produces one number. This explains the dimensions rather than asking you to memorize a multiplication rule. Some libraries use row-vector notation and store a transposed matrix; follow the stated convention.

What is chosen, learned and computed?

We choose input and output widths. Training learns weights and biases. A forward pass computes output numbers. Tensor axes such as `[B, T, D]` mean batch size, token positions and features only when the program defines them that way. A shape alone does not tell you an axis's meaning.

Worked example: follow one row at a time

```
x = [2.0, 1.0]
weights = [[3.0, -1.0], [0.0, 2.0]]
biases = [1.0, 0.0]
y = [sum(a*b for a, b in zip(x, row)) + bias
     for row, bias in zip(weights, biases)]
print(y)  # [6.0, 2.0]
```

The first weight row is `[3, -1]`: its dot product with `[2, 1]` is `3*2 + (-1)*1 = 5`; adding bias `1` gives `6`. The second row gives `0*2 + 2*1 + 0 = 2`. Two rows produce two output numbers. `zip` pairs corresponding entries, and `sum` adds their products.

This code supplies the weights manually. It performs a forward calculation; no training happens. Changing one weight changes its contribution, not the layer's dimensions.

Check your understanding

Question: Why does a matrix with three rows and two columns map a length-two vector to three numbers?

Answer: Each row has two weights, so it combines the two input values into one output. Three separate rows perform three such combinations. Mismatched dimensions mean the required pairings are undefined; Python's `zip` can silently truncate mismatched lists, so real code should check widths.

3. Scores become probabilities: softmax and temperature

The concept: scores need a consistent interpretation

A model's final numerical scores are called **logits**. They can be negative, positive or very large. To sample one candidate, we need nonnegative probabilities that add to one. Softmax is a particular rule for converting relative scores into that distribution.

A probability describes the model's distribution over candidates. It is not a measurement that a statement is factually correct. That requires evaluation against the world or a reliable target.

The formula and every symbol

$$p_i = \frac{\exp(z_i/T)}{\sum_{j=1}^{V} \exp(z_j/T)}$$

z_i is candidate i's logit, p_i its probability, V the number of candidates, and $T > 0$ the temperature. j is an index used to add over all candidates. $\exp(a) = e^a$ is the exponential function; it is always positive.

Why this formula has these parts

First choose positive relative weights: exponentiating a score makes every weight positive. Next divide each weight by their total. The resulting fractions add to one because the numerators collectively equal the denominator.

Why an exponential specifically? It makes a difference in scores correspond to a ratio of probabilities:

$$\frac{p_i}{p_j} = \exp((z_i - z_j)/T)$$

The shared denominator cancels. A larger score difference produces a larger odds ratio. This motivates softmax; positivity alone would allow other transformations, so softmax is a modeling choice rather than the only possible normalization.

Temperature divides the score differences. Smaller positive T amplifies preferences; larger T flattens them. Temperature is normally an inference setting, not a learned weight. “Temperature zero” in software generally means choose a maximum-scoring candidate directly; substituting zero into this formula would divide by zero.

Derive the stable implementation

Exponentiating large logits can overflow. Subtract their maximum m before exponentiating. Every exponential is multiplied by the same factor $\exp(-m)$, which cancels between numerator and denominator. Therefore the probabilities stay the same while the largest exponential becomes one.

Worked example: calculate before sampling

```
import math

def softmax(scores, temperature=1.0):
    scaled = [s / temperature for s in scores]
    largest = max(scaled)
    values = [math.exp(s - largest) for s in scaled]
    return [v / sum(values) for v in values]

print([round(p, 3) for p in softmax([2, 1, 0])])
print([round(p, 3) for p in softmax([2, 1, 0], 2)])
```

At temperature `1`, logits `[2, 1, 0]` become approximately `[0.665, 0.245, 0.090]`. At temperature `2`, the same ranking becomes approximately `[0.506, 0.307, 0.186]`. No knowledge changed: the distribution spread out. The function returns probabilities; a later sampling or selection operation chooses an output.

Check your understanding

Question: If we add 1,000 to every logit, must the probabilities change?

Answer: No. All pairwise score differences stay the same, and the common exponential factor cancels. Subtracting the maximum uses exactly this property. Lowering temperature can make a wrong leading choice more dominant, so confidence and correctness must be checked separately.

4. Build your first language predictor with counts

The concept: learning can mean estimating a frequency

Language prediction asks which token might come next given previous tokens. A **bigram model** makes a simplifying assumption: use only the immediately preceding token as context. A token can be a character in this lesson; it need not be a complete word.

The model learns a table of transition counts from text. At prediction time it looks up the row for the current token. This already separates training, which builds counts, from inference, which uses the table.

Derive the probability from counting

Let $C(i, j)$ count how often token j immediately follows token i. Let $N_i = \sum_k C(i, k)$ count all observed transitions out of i. Then, when $N_i > 0$:

$$P(j \mid i) = \frac{C(i, j)}{N_i}$$

$P(j \mid i)$ means "probability of next token j, given previous token i." The vertical bar means "given," not division. k indexes the possible next tokens. We divide by the row total because we are comparing outcomes under that particular context. Dividing by the entire corpus length would answer a different question.

Adding these fractions over all next tokens gives $N_i / N_i = 1$. The normalization therefore produces a valid distribution. The assumption is that observed relative frequency is a useful estimate for future inputs from a comparable source.

Why smoothing changes both numerator and denominator

A zero count assigns probability zero even when an event is merely unseen. Choose a vocabulary containing V possible next tokens and add a positive pseudo-count α to **each** candidate:

$$P(j \mid i) = \frac{C(i, j) + \alpha}{N_i + \alpha V}$$

The denominator grows by αV because we added α exactly V times. α is chosen; counts come from data. Smoothing is an assumption about how to distribute uncertainty, not newly observed evidence. For a completely unseen context, this version gives a uniform distribution over the chosen vocabulary.

Worked example: inspect the actual transitions

```
from collections import defaultdict, Counter
text = "hello hello"
counts = defaultdict(Counter)
for previous, following in zip(text, text[1:]):
    counts[previous][following] += 1

row = counts["l"]
total = sum(row.values())
print(dict(row))
print({char: count / total for char, count in row.items()})
```

`zip(text, text[1:])` pairs each character with its successor. In `hello hello`, the character `l` is followed by `l` twice and by `o` twice. Its row total is four, giving probability `0.5` to each observed successor. This lab implements the unsmoothed rule; it prints only observed successors, with unobserved transitions implicitly assigned zero.

To implement smoothing, enumerate the complete chosen vocabulary, add α to every count, and use the adjusted total. Adding only to observed entries would leave unseen entries at zero.

Check your understanding

Question: Why can this model fail even if every count is calculated correctly?

Answer: Different earlier sentences can end with the same token. The model uses the same transition row for both because it discarded earlier context. Correct counting cannot recover information excluded by the architecture. A larger context can distinguish more situations, but also creates more contexts with little or no data.

5. Turn text into tokens, then into embeddings

The concept: text becomes indices before it becomes vectors

A neural model operates on numbers. **Tokenization** converts text into a sequence of vocabulary entries and their integer IDs. **Embedding lookup** converts those IDs into learned vectors. These are separate operations, with different purposes.

An ID identifies a vocabulary entry. Its numerical size is not a measure of meaning: token 100 is not necessarily twice as meaningful or more similar to token 50. Treating IDs as quantities would impose an arbitrary relationship on the vocabulary.

The rule and what each symbol means

Let V be vocabulary size and D the embedding width. The embedding table E has shape `[V, D]`. For token ID t_i at position i:

$$h_i = E[t_i, :]$$

Read this as “select row t_i, taking all its columns.” The colon means all feature coordinates. h_i is a length-D vector, not another ID. A length-T sequence of IDs becomes a `[T, D]` array of vectors; a batch becomes `[B, T, D]`.

Why lookup is the right operation

Conceptually, represent a token as a length-V vector containing one at its ID and zero elsewhere. Multiplying that vector by E selects exactly one row, since every other row is multiplied by zero. Direct row lookup gives the same result without constructing the mostly-zero vector.

Training adjusts the selected embedding rows through gradients from the downstream loss. Tokenization itself is a defined encoding process, often constructed separately before model training. We choose or inherit the vocabulary, tokenizer rules, embedding width and compatible checkpoint. Token IDs must use the exact mapping expected by that checkpoint.

Subwords and chat formatting

A fixed word vocabulary cannot contain every name and spelling. Subword or byte-based tokenizers can compose many strings from smaller pieces; exact behavior depends on the tokenizer. Splitting on spaces is only a teaching approximation.

A chat template serializes roles, boundaries and messages into the model's expected input. Extra role markers consume tokens too. The input contract therefore includes the tokenizer and template, not just the visible sentence.

Worked example: encoding is not embedding

```
text = "call maya"
vocabulary = sorted(set(text))
to_id = {char: i for i, char in enumerate(vocabulary)}
ids = [to_id[char] for char in text]
recovered = "".join(vocabulary[i] for i in ids)
print(ids)
print(recovered)
assert recovered == text
```

This code builds a character vocabulary from one string, sorts it for a consistent ordering and maps each character to an ID. The space is also a character. It then decodes the IDs to recover the original text. The assertion checks the round trip.

There is **no embedding table or trained model in this lab**. To add the next stage, create a `[V, D]` table and look up a row for each ID. Arbitrary initial rows illustrate lookup, but useful representations require learning. Do not rebuild a new ID mapping for each request in a deployed model.

Check your understanding

Question: Are the model's internal token vectors automatically suitable for document search?

Answer: No. Search needs a representation and scoring contract trained or validated for comparing whole queries and documents. A token lookup is a building block. A sentence embedding usually involves further processing and aggregation. Equal vector lengths do not make independently trained coordinate systems compatible.

6. Loss and gradients: teach one number to improve

The concept: prediction, error and learning are different operations

A tiny model is a calculation with an adjustable number. We supply its structure; learning adjusts the number. It does not invent the multiplication rule during this exercise.

Use the model $\hat{y} = xw$. x is the input, w is the learned weight, and $\hat{y}$ is the prediction. y is the correct training target. At inference, multiplying the input by the current weight produces the answer. The target is needed to score training, not to calculate a prediction.

Derive the loss before inserting numbers

First define the signed error $e = \hat{y} - y$. Positive and negative errors can cancel if we simply add them across examples. One useful choice is to square each error:

$$L = (\hat{y} - y)^2 = (xw - y)^2$$

L is the loss for one example. Squaring makes it nonnegative and penalizes larger errors more strongly. Squared error is a chosen objective, not a universal requirement. Absolute error or other objectives produce different gradients. In Python, `**2` means “square”; `^` is not the exponentiation operator.

What is a gradient, before we calculate one?

The derivative dL/dw measures how fast the loss changes as we make a tiny change in w, keeping the input and target fixed. With one weight it is a slope. With many weights, the **gradient** collects one such partial derivative for each weight.

For a tiny change Δw, the loss change is approximately $g\Delta w$, where $g = dL/dw$. This is a local approximation. Positive g suggests decreasing w to reduce loss; negative g suggests increasing w. A zero gradient does not generally prove a global minimum, although it does identify the minimum of this simple quadratic when $x \neq 0$.

Derive the gradient using the chain rule

Follow the dependency chain: **weight → prediction → error → squared error**.

1. Prediction changes at rate x with respect to w, because $\hat{y} = xw$.
2. Error changes at rate one with respect to prediction, because the target is held fixed.
3. Squared error changes at rate $2e$ with respect to error.

Multiply these rates along the chain:

$$g = \frac{dL}{dw} = 2e \times 1 \times x = 2(xw - y)x$$

Why does differentiating a square give $2e$? Expand a small change: $(e + \delta)^2 - e^2 = 2e\delta + \delta^2$. Divide by δ to get $2e + \delta$. As δ approaches zero, the rate approaches $2e$. That explains the rule instead of requiring you to memorize it.

The factor outside the brackets is **not always four**. It is $2x$ here. A different prediction rule, input or loss can change the derivative.

Why the learning rate is needed

A gradient gives a local slope, not the best next weight or a safe distance to travel. We choose a positive **learning rate** η to scale the update:

$$\Delta w = -\eta g, \qquad w_{new} = w_{old} - \eta g$$

Subtracting the gradient gives a locally downhill direction. Multiplying by η controls the distance. The actual step is $-\eta g$; the learning rate is the multiplier. Even a fixed learning rate gives different steps as the gradient changes. Schedules can also change the rate over training.

Very small rates can learn slowly. Large rates can overshoot or diverge because a local slope is not a reliable guide for an arbitrarily long jump. A real optimizer may use momentum or adaptive scaling; this lesson uses plain gradient descent.

Worked example: now substitute the lesson's numbers

Our single training pair is `x = 2`, `y = 6`. Substituting gives:

$$\hat{y} = 2w, \qquad L = (2w - 6)^2, \qquad g = 4(2w - 6)$$

The `2` in the prediction comes from the supplied input. The `6` comes from the target. The `4` in the gradient comes from `2` for the square times `2` for the input. With `x = 3` instead, the gradient would be `6*(3*w - y)`.

At `w = 1`, prediction is `2`, error is `-4`, loss is `16`, and gradient is `-16`. With learning rate `0.1`, the update is `1 - 0.1*(-16) = 2.6`. The new prediction is `5.2`; the new loss is `0.64`.

```
w = 1.0
learning_rate = 0.1
for step in range(5):
    prediction = 2 * w
    loss = (prediction - 6) ** 2
    gradient = 4 * (prediction - 6)
    print(step, round(w, 4), round(loss, 4))
    w -= learning_rate * gradient
```

Each printed line shows the weight and loss **before** that iteration's update. Rounding can display zero even when a small nonzero loss remains. The line `gradient = 4 * (prediction - 6)` uses the derivative we just derived. The target remains fixed while the loop adjusts only `w`.

Derive when this particular example overshoots

For this example, the best weight is `3` and the gradient is `8*(w-3)`. Subtract three from both sides of the update:

$$w_{new} - 3 = (1 - 8\eta)(w_{old} - 3)$$

The distance from the optimum shrinks when $|1 - 8\eta| < 1$, giving $0 < \eta < 0.25$. At `0.125` it reaches the minimum in one step. Between `0.125` and `0.25`, it crosses the minimum but still gets closer. At `0.25`, it oscillates at the same distance. This bound belongs to **this quadratic and this optimizer**; it is not a universal recommended learning rate.

Check your understanding

Question: Which parts are chosen, supplied, learned and derived?

Answer: We choose the multiplication model, squared-error objective and learning rate. Data supplies the input and target. Training changes the weight. Calculus derives the gradient from the model and loss. We can solve this one-weight example directly, but real networks require iterative optimization because their interacting parameters are much harder to solve for.

7. The training loop, batches and honest evaluation

The concept: training optimizes a measured objective

Training repeats a sequence: select data, predict, measure error, differentiate and update parameters. A **batch** is the set of examples processed for one calculation. An **epoch** is one pass through the training dataset. A **step** usually means one optimizer update; it need not mean one example or one epoch.

The objective must match the output. Lesson 6 predicted a number and used squared error. A language model predicts a distribution over next tokens, so we usually score how much probability it assigns to each correct next token.

Derive the next-token loss

For a sequence of correct tokens, the model assigns a conditional probability to each token given the earlier prefix. The probability of the sequence is the product of those conditional probabilities. Maximizing this product encourages the correct sequence.

Products of small probabilities become extremely small. A logarithm converts a product into a sum. Negating the log turns maximization into minimization. Therefore the penalty at one position is $-\ln p_t$, where p_t is the probability assigned to the correct target at position t. $\ln$ means the natural logarithm.

Average the penalties over the N positions we actually score:

$$L = -\frac{1}{N}\sum_{t=1}^{N} \ln p_t$$

L is mean cross-entropy for one-hot next-token targets. We add token penalties and divide by their count to control the scale as batch sizes or sequence lengths change. This is not a percentage accuracy. Probability one gives zero penalty; a correct target assigned very small probability gives a large penalty.

Perplexity is $\exp(L)$ for this mean natural-log loss. Compare it only with the same prediction units and scoring setup; a changed tokenizer changes the units.

Derive the training loop from those dependencies

The prediction depends on current weights, and loss depends on that prediction. A backward pass applies the chain rule to calculate weight gradients. The optimizer then changes the weights, making a new forward pass necessary for the next step.

In PyTorch, the main operations are `zero_grad`, forward computation, loss computation, `backward`, and `step`. Gradients accumulate by default, so clear them before a fresh update unless intentionally accumulating. You may clear before the forward pass or before `backward`; what matters is the

intended accumulation boundary. `backward` calculates gradients; `step` applies the optimizer's update.

For language modeling, the input tokens and targets are shifted: a position sees the prefix and predicts the following token. A causal mask prevents using later inputs. A **loss mask** separately decides which targets count toward the objective. Excluding prompt targets from the loss does not remove the prompt from the model's input.

What is chosen versus learned?

Batch size, learning rate, epochs, optimizer and data splits are chosen. Weights are learned. Loss and evaluation metrics are computed. With equally sized microbatches averaged consistently, effective batch size is microbatch size times accumulation steps times data-parallel workers. Unequal scored-token counts require appropriate weighting.

Worked example: training success is not generalization

```
train = [("remind me at six", "reminder"),
         ("save this thought", "note")]
test = [("could you nudge me at six", "reminder"),
        ("save this thought", "note")]
def predict(text):
    return "reminder" if "remind" in text else "note"
def accuracy(rows):
    return sum(predict(x) == y for x, y in rows) / len(rows)
print(accuracy(train), accuracy(test))  # 1.0 0.5
```

This lab trains no weights; it evaluates a fixed rule to isolate the evaluation idea. The rule gets both training cases right, but misses the paraphrased reminder in the test set, giving `1.0` and `0.5`. The repeated note appears in both sets, so this is a demonstration, not a clean independent benchmark.

An honest experiment splits by the source of possible leakage: document, conversation, person or time as appropriate. Use training data for fitting, validation for choosing settings, and a held-out test for final assessment. Repeatedly selecting changes using a test score gradually turns that test into a validation set.

Check your understanding

Question: Why can training loss fall without useful performance improving?

Answer: The model can fit patterns specific to the training data, including duplicates or accidental shortcuts. Independent evaluation measures whether useful behavior transfers. A low next-token loss also does not prove that application actions are correct or authorized.

8. Neural networks and autograd, one layer at a time

The concept: composition needs a nonlinear operation

A linear layer mixes features using multiplication and addition. Stacking such layers without any nonlinear operation does not create a more expressive family of mappings: the stack can be rewritten as one affine layer. A **nonlinearity** lets different input regions follow different relationships.

A **neural network** composes layers with learned parameters. A **hidden representation** is an intermediate result rather than the final output. Training adjusts the parameters of these connected operations using derivatives of the final loss.

Derive why linear layers alone collapse

For a scalar input, let $h = ax + b$ and $\hat{y} = ch + d$. Substitute the first expression into the second:

$$\hat{y} = c(ax + b) + d = (ca)x + (cb + d)$$

a, b, c, d are layer parameters. The whole expression is still one multiplication plus an offset. Matrix multiplication has the same composition property. Extra linear layers can change parameterization, but alone they do not add the input-dependent bends we need.

The nonlinear rule and its gradient

ReLU, the rectified linear unit, is:

$$\mathrm{ReLU}(z) = \max(0, z)$$

z is a layer's pre-activation value. Negative inputs become zero; positive inputs pass through. Its derivative is zero for negative z and one for positive z. At zero it is not classically differentiable; implementations choose a convention. A hidden unit can now switch its contribution on or off depending on the input.

For a network $z = wx + b$, $h = \mathrm{ReLU}(z)$, $\hat{y} = vh + c$, the gradient with respect to w multiplies the local rates along the path:

$$\frac{dL}{dw} = \frac{dL}{d\hat{y}} \times v \times \mathrm{ReLU}'(z) \times x$$

L is the final loss, and the prime means derivative. The same chain rule from Lesson 6 works across more operations. If a value affects the loss through several paths, add their gradient contributions. For $a = x^2$ and $L = a + a$, both branches contribute, giving $dL/dx = 4x$.

What autograd actually does

Automatic differentiation records differentiable operations and evaluates their local derivatives in reverse dependency order. It applies the chain rule efficiently; it does not choose the loss or guess the correct answer. The optimizer separately uses the resulting gradients to update parameters. PyTorch's autograd tutorial documents this distinction and gradient accumulation.

Normalization and regularization have different purposes. Normalization manages numerical scales inside the calculation. Regularization constrains fitting to encourage generalization; dropout and weight decay are common choices. None replaces evaluation on independent data.

Worked example: a network with manually supplied weights

```
def relu(x):
    return max(0.0, x)

def network(x):
    hidden = [relu(2*x - 1), relu(-x + 2)]
    return hidden[0] - hidden[1]

for x in [0, 1, 2, 3]:
    print(x, network(x))  # -2, 0, 3, 5
```

At `x = 0`, the two pre-activations are `-1` and `2`. ReLU gives `[0, 2]`, and subtracting the second hidden feature from the first produces `-2`. At `x = 2`, they are `3` and `0`, producing `3`. The output rule changes across regions because different units are active.

The lab performs inference only. There is no optimizer or target. To train it, we would supply targets, define a loss, differentiate and update the listed coefficients.

Check your understanding

Question: Does calling `backward()` train the model by itself?

Answer: It calculates gradients. An optimizer update changes parameters. Separating these operations makes gradient accumulation and debugging possible. Check which parameters require gradients, which received gradients and which belong to the optimizer when learning appears stuck.

9. Attention: let one word consult the others

The concept: representations need information from context

A token's meaning depends on other tokens. Attention computes a context-dependent mixture of information from visible positions. It does this numerically, using learned projections rather than a hand-written rule naming which word should consult which other word.

Each token representation is projected into a **query** (what to match), a **key** (what can be matched), and a **value** (the information available to mix). These roles are useful interpretations of three learned vector transformations, not guaranteed human-readable fields.

Build the rule from one query

For a query vector q and key k_j, a dot product sums coordinate-wise products into a match score. With key width d_k:

$$s_j = \frac{q \cdot k_j}{\sqrt{d_k}} + m_j, \qquad a_j = \frac{\exp(s_j)}{\sum_i \exp(s_i)}, \qquad z = \sum_j a_j v_j$$

j indexes candidate positions; v_j is that position's value vector; a_j is its nonnegative mixing weight. z is the output vector. The mask value m_j is zero for allowed positions and negative infinity for disallowed ones. After softmax, disallowed positions get zero weight. At least one position must be allowed in each row.

Why scale by the square root? Under a simplifying assumption of independent, zero-mean, unit-variance coordinates, a sum of d_k products has variance d_k. Dividing by $\sqrt{d_k}$ brings the variance back to about one. This reduces the tendency for wider keys alone to make softmax extremely concentrated. It is a scale argument, not a claim that all trained coordinates remain independent.

Derive the matrix formula and its shapes

Stack queries, keys and values into Q, K, V. For one head over T positions, Q and K have shape `[T, d_k]`, and V has shape `[T, d_v]`. Then:

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right), \qquad Z = AV$$

The superscript $\top$ means transpose: swap matrix axes. `Q @ K.T` produces `[T, T]` scores comparing each query with every key. Softmax operates separately across each row's key positions. Multiplying `[T, T]` attention weights by `[T, d_v]` values gives `[T, d_v]` mixtures.

$Q = HW_Q$, $K = HW_K$, and $V = HW_V$ use learned projection matrices on hidden states H in row-vector notation. We choose head counts and widths; training learns those projection weights; the attention weights A are computed fresh from the input. They are not the same thing as learned model parameters.

Causality and multiple heads

A decoder's causal mask prevents a position from reading future input positions. Multiple heads perform several such mixtures with distinct projections, then combine them. Heads provide representational capacity; there is no guarantee that each learns one tidy linguistic job.

Worked example: scores, weights and values are different

```
import math
scores = [0.0, math.log(3)]
values = [[2.0, 0.0], [0.0, 4.0]]
exp_scores = [math.exp(s) for s in scores]
weights = [s / sum(exp_scores) for s in exp_scores]
output = [sum(weights[i] * values[i][j] for i in range(2))
          for j in range(2)]
print(weights)  # [0.25, 0.75]
print(output)   # [0.5, 3.0]
```

The supplied scores `0` and `ln(3)` exponentiate to `1` and `3`, so their weights are `1/4` and `3/4`. The first output coordinate is `0.25*2 + 0.75*0 = 0.5`; the second is `0.25*0 + 0.75*4 = 3`. This lab starts with scores; it does not calculate learned Q/K projections.

Check your understanding

Question: Why not return just the scores?

Answer: Scores describe where to draw information from. Weighted values carry that information. Test a causal implementation by altering a future token and verifying earlier-position outputs remain unchanged. A decreasing training loss alone cannot detect future-token leakage.

10. Assemble a tiny transformer and follow one token

The concept: a decoder refines a representation repeatedly

A decoder-only transformer turns token IDs into contextual feature vectors, then into scores for the next token. Attention exchanges information across visible positions. A feed-forward network transforms features within each position. Repeating these operations lets later layers work with representations built by earlier layers.

The architecture defines the operations. Training learns the embedding, projection, normalization and feed-forward parameters. One forward pass computes representations and scores using those current parameters.

The block rule and each symbol

A common **pre-normalization** block can be written as:

$$U = H + \mathrm{Attention}(\mathrm{Norm}(H))$$

$$H_{next} = U + \mathrm{MLP}(\mathrm{Norm}(U))$$

H is the table of hidden vectors entering the block. `Norm` normalizes features within each position. `Attention` includes the head combination and output projection. `MLP` means a feed-forward neural network. U is an intermediate result; H_{next} feeds the following block. This is one architecture pattern, not a formula every transformer must share.

Why add the input back?

The additions are **residual connections**. A sublayer predicts a modification to an existing representation. If the modification is small, the original can pass through largely intact. The addition also provides a direct path in the derivative, helping information and gradients propagate through a deep composition. Shapes must match before values can be added coordinate by coordinate.

The feed-forward sublayer usually expands the feature width, applies a nonlinearity and projects back. Expanding supplies intermediate capacity; projecting back restores the width required by the residual addition. Without a nonlinearity, the two linear maps would collapse into one, as in Lesson 8.

Derive normalization from a scale measurement

For a length-D vector x, root-mean-square normalization first squares coordinates, averages them and takes the square root:

$$\mathrm{RMSNorm}(x)_i = g_i \frac{x_i}{\sqrt{\frac{1}{D}\sum_{j=1}^{D} x_j^2 + \epsilon}}$$

g_i is a learned scale and $\epsilon > 0$ prevents division by zero. The denominator estimates the vector's coordinate scale; dividing controls that scale. LayerNorm additionally subtracts the coordinate mean and uses centered variance, often with learned scale and offset. The workshop uses LayerNorm. These techniques affect numerical behavior; they do not certify correctness or replace regularization.

Derive the output shape

For batch size B, sequence length T and hidden width D, the hidden tensor has shape `[B, T, D]`. A final learned projection produces one score for each of V vocabulary entries, changing the last axis to `[B, T, V]`. At position t, those scores predict the token after its visible prefix. Softmax or a selection procedure interprets the scores; the projection does not itself execute an action.

Worked example: a shape contract before a full network

```
batch, positions, features, vocabulary = 2, 4, 8, 10
ids = [[1, 3, 2, 0], [2, 1, 4, 3]]
logits_shape = (len(ids), len(ids[0]), vocabulary)
print(logits_shape)  # (2, 4, 10)
print("Scores for one final position:", vocabulary)
```

This lab reports `(2, 4, 10)` because there are two sequences, four positions each and ten vocabulary candidates per position. It calculates no embeddings or transformer output. Its purpose is to make the output interface explicit before you read the complete implementation in Lesson 30.

Decoder-only models predict continuations under a causal mask. Encoder-only models commonly build representations with bidirectional context. Encoder-decoder models use a source encoder and an output decoder. Select the actual architecture rather than applying one diagram to every model.

Check your understanding

Question: Why does the vocabulary projection change feature width but preserve sequence length?

Answer: It applies the same feature-to-score map separately at every existing position. Creating a new token happens later: select a next token, append its ID and run the next generation step.

11. Position, context limits and the KV cache

The concept: order and reuse solve different problems

Attention needs information about position to represent order. A **context limit** bounds how many positions a model/runtime supports in a request. A **KV cache** stores previously computed keys and values so generation can reuse them. These are related to sequence length, but they are different mechanisms.

Without position information, unmasked self-attention treats a permutation of the input as the same permutation of its outputs. Causal masking adds directional structure, but it is not a replacement for a model's trained positional scheme.

The position rule

One approach adds a learned position vector to each token vector. Another, **rotary position embedding**, rotates pairs of query/key coordinates by an angle depending on position and frequency. The ordinary two-dimensional rotation is:

$$a' = a\cos\theta - b\sin\theta, \qquad b' = a\sin\theta + b\cos\theta$$

a, b are a pair of original coordinates, a', b' the rotated coordinates, and θ the position-dependent angle. Rotation preserves length. Applying position-dependent rotations to queries and keys makes their dot product depend on relative angle, hence relative position. Actual rotary implementations use many pairs and frequency rules. Reusing a checkpoint requires its corresponding position configuration.

Why caching is possible during generation

In causal attention, earlier positions cannot see later ones. Appending a token therefore does not change the earlier key/value representations in a conventional causal decoder with a consistent prefix and positional scheme. Store those K/V tensors and compute the new token's query, key and value at each layer.

Prefill processes the prompt and builds the initial cache. **Decode** processes new tokens using it. Each new query still reads prior keys and values: the cache avoids recomputing their representations, not all attention work. A KV cache is runtime numerical state, not a searchable memory of user facts.

Derive the memory formula by counting elements

At each layer and KV head, each cached token needs a key vector and a value vector. If their widths are both d_h:

$$M_{KV} \approx 2BLTH_{KV}d_h s$$

B is sequence count, L layer count, T cached tokens per sequence, H_{KV} KV heads, d_h coordinates per head, and s bytes per coordinate. The factor two counts K and V. Multiplying counts of stored elements by bytes per element gives bytes; this is storage accounting, not a learned formula.

Grouped-query attention shares K/V across query heads, so the formula counts **KV heads**, not query heads. It assumes these dimensions and precisions are uniform. Variable lengths, allocation blocks and implementation details can change actual allocation. Model weights and temporary buffers are additional memory.

Worked example: calculate the cache separately from weights

```
batch, layers, tokens = 1, 28, 1024
kv_heads, head_width, bytes_per_number = 8, 128, 2
memory = (2 * batch * layers * tokens * kv_heads
          * head_width * bytes_per_number)
print(memory / 1024**2, "MiB")  # 112.0 MiB
```

The illustrative dimensions yield `112 MiB`, dividing bytes by `1024**2`. Doubling tokens doubles this element count; doubling bytes per element does too. The two-byte assumption describes these stored elements, not necessarily the precision used for model weights.

Check your understanding

Question: If a request fits inside the context limit, is all its information used reliably?

Answer: No. Fitting establishes a length constraint, not reliable retrieval or reasoning. Relevant information can still be overlooked. Measure answer quality across positions and lengths, and reserve capacity for generated output under the runtime's context rules.

12. Choose output tokens and understand inference optimizations

The concept: token selection and computation speed are separate choices

A forward pass gives next-token scores. A **decoding policy** chooses a token from them. An **execution optimization** changes how the calculation is performed. To understand an inference setting, first identify which of those jobs it changes.

Greedy selection picks a maximum-scoring token. Sampling draws from a probability distribution, often modified by temperature or filters. Greedy selection maximizes the immediate score, not necessarily the probability or usefulness of the complete answer. Beam search keeps several candidate continuations, with its own scoring and resource tradeoffs.

Derive top-p sampling

Let sorted candidate probabilities satisfy $p_1 \geq p_2 \geq \ldots$. Choose a threshold τ between zero and one. Keep the smallest prefix whose cumulative probability reaches it:

$$m = \min \left\{ r : \sum_{i=1}^{r} p_i \geq \tau \right\}$$

m is the retained candidate count, and r is a possible prefix length. This is **top-p**, also called nucleus sampling. The threshold controls retained mass, not a fixed candidate count. Top-k instead retains a chosen number of candidates.

After removing the rest, divide each retained probability by retained total S:

$$p'_i = p_i / S, \qquad S = \sum_{j=1}^{m} p_j$$

The division is necessary because the remaining probabilities initially sum to S, which may be less than one. Sampling from the normalized remainder then produces one token. Filtering or temperature changes can alter behavior; they do not supply missing factual evidence.

Why the speed techniques work

A KV cache reuses past keys and values as explained in Lesson 11. **FlashAttention** reorganizes exact attention using tiled computation and reduced traffic between memory levels; its goal is efficient execution of the attention operation, subject to normal numerical differences. It is not a token-selection policy. See the original FlashAttention paper.

Speculative decoding uses a cheaper proposal process and a target-model verification/acceptance procedure. Proper exact sampling variants preserve the target distribution using correction steps; simply accepting plausible draft tokens is not that algorithm. Gains depend on acceptance and runtime

costs.

A **mixture-of-experts** architecture routes representations through a subset of expert networks. Sparse activation can reduce active work relative to using all experts, but total parameter storage and communication remain relevant. It changes the architecture; it is not a drop-in sampling rule.

Worked example: retain, normalize, then sample

```
probabilities = [0.60, 0.25, 0.10, 0.05]
top_p = 0.80
chosen, total = [], 0.0
for index, probability in enumerate(probabilities):
    chosen.append(index)
    total += probability
    if total >= top_p:
        break
print(chosen)  # [0, 1]
print([probabilities[i] / total for i in chosen])
```

The input probabilities are already sorted. Starting with `0.60`, adding `0.25` reaches `0.85`, exceeding threshold `0.80`. Keep the first two candidates. Their normalized probabilities become approximately `0.706` and `0.294`. The lab reports the retained set and probabilities; it does not actually draw a sample.

With threshold `0.95`, three candidates suffice. With a fixed top-k value of two, two would always remain regardless of their total mass. That difference explains why the policies behave differently as model confidence changes.

Check your understanding

Question: What stops generation?

Answer: A configured stop condition such as an end token, output-token budget, deadline or application rule. Stopping and token selection are explicit parts of the generation loop. Faster attention does not automatically improve the answer, and a more varied sample does not automatically improve task success.

13. Prompting, context and JSON that deserves checking

The concept: define the input contract and the output contract

A **prompt** supplies instructions and information to a model during inference. It changes the input, not normally the weights. **Context engineering** selects and arranges the information available within the request's length budget. **Structured output** asks for a machine-readable shape so application code can process the response.

These mechanisms address different problems. Clear instructions describe the task. Relevant evidence supplies missing facts. A schema defines fields and types. Application validation establishes whether a particular proposed result is usable.

Derive the context budget

A request has a finite capacity. Count every component in the tokenizer used by the model:

$$I + H + E + O \leq C$$

I is instruction and formatting tokens, H conversation-history tokens, E evidence tokens, O reserved output tokens, and C the combined context capacity under this simplified accounting. Each component must be counted once. Subtracting the other components gives the available evidence budget: $E \leq C - I - H - O$.

The inequality comes from adding lengths, not from model training. Runtime APIs can impose additional input/output limits. Budgeting ensures the request fits; it does not ensure the model pays attention to every included fact. Prefer relevant, traceable evidence over arbitrary extra text.

Derive the validation sequence

A string becomes actionable only after several distinct checks:

1. **Parse:** Is it valid JSON?
2. **Schema:** Are required fields present with appropriate types and permitted values?
3. **Meaning:** Are values sensible for this task and supported by the input?
4. **Authority:** May this user perform this exact action on this resource?
5. **Execution:** Did the external operation actually succeed?

Passing an earlier check cannot imply a later one, because later checks require additional information. A syntactically valid date string cannot establish that the user requested that date or owns the destination calendar.

Constrained decoding can restrict token choices to produce outputs matching a supported grammar/schema. It reduces certain formatting failures; it cannot supply missing source facts or user permission. Treat retrieved content as evidence rather than new authority over the application.

Worked example: valid JSON with an invalid value

```
import json
raw = '{"intent":"reminder","hour":25}'
obj = json.loads(raw)
shape_ok = isinstance(obj.get("hour"), int)
meaning_ok = shape_ok and 0 <= obj["hour"] <= 23
print(shape_ok, meaning_ok)  # True False
```

`json.loads` parses the string. `shape_ok` checks that `hour` is a Python integer; `meaning_ok` separately checks the allowed range. An hour of 25 passes the type check and fails the range check. A real schema can enforce the range too; it still cannot prove that the chosen hour matches the request.

This is a small demonstration rather than complete JSON validation: in Python, booleans are also instances of `int`, and the code does not check every field. Production validation should define the precise contract.

Check your understanding

Question: What should you change when a model produces valid fields but uses the wrong source fact?

Answer: Inspect evidence selection and the interpretation of that evidence. More formatting instructions may not fix a missing or conflicting fact. Compare one prompt change at a time on the same cases, measuring both task correctness and formatting rather than assuming a longer prompt is better.

14. Embeddings and search: find meaning without exact words

The concept: retrieval ranks representations

An **embedding** represents an item as a vector. Semantic retrieval compares a query vector with document vectors to find likely useful candidates. Its quality depends on what the representation was trained to preserve and how we measure similarity.

A vector database adds storage, filtering, indexing and update behavior. It does not make arbitrary vectors semantically compatible. Queries and documents must follow the intended shared representation contract, including compatible encoders, preprocessing and normalization.

Derive cosine similarity

A dot product sums coordinate-wise products. It depends on both vector direction and magnitude. To compare direction, normalize each nonzero vector to length one before taking the dot product:

$$\cos(q, d) = \left(\frac{q}{\|q\|}\right) \cdot \left(\frac{d}{\|d\|}\right) = \frac{q \cdot d}{\|q\|\|d\|}$$

q is the query vector and d the document vector. The length of q is $\|q\| = \sqrt{\sum_i q_i^2}$. Double bars mean length, not absolute value of each coordinate. The numerator measures alignment; dividing by both lengths removes their magnitude contribution.

For real nonzero vectors, cosine ranges from minus one to one. Larger means more aligned. It is a similarity score, not a probability of relevance. Zero vectors require an explicit handling rule because their length is zero. A higher score is useful only if the embedding model makes useful items align.

What is chosen, learned and computed?

We choose an embedding model and retrieval metric. The encoder's training established its parameters. Inference computes vectors, and search computes scores or approximates a ranking. **Exact search** scores every eligible vector. An **approximate nearest-neighbor index** reduces work by narrowing the candidates, potentially missing some exact neighbors.

This gives a diagnostic baseline: compare approximate results with exact results on the same vectors and eligible documents. If both rankings are poor against relevance labels, investigate representation, data or task fit. If approximate retrieval loses good exact matches, investigate index settings. Higher approximate recall can cost more time or memory.

Worked example: calculate direction explicitly

```
import math

def cosine(a, b):
    dot = sum(x*y for x, y in zip(a, b))
    length_a = math.sqrt(sum(x*x for x in a))
    length_b = math.sqrt(sum(x*x for x in b))
    return dot / (length_a * length_b)

print(round(cosine([1, 0], [1, 1]), 3))  # 0.707
print(round(cosine([1, 0], [3, 4]), 3))  # 0.600
```

For `[1,0]` and `[1,1]`, the dot product is `1`; lengths are `1` and `sqrt(2)`, yielding about `0.707`. For `[3,4]`, length is `5`, giving `3/5 = 0.6`. The first candidate is more aligned with the query.

Multiplying a candidate vector by a positive constant leaves its cosine unchanged because numerator and denominator scale equally. An unnormalized dot product does change, so metric choice matters.

Check your understanding

Question: Is the closest vector necessarily a usable answer source?

Answer: No. Similarity does not establish relevance, freshness, truth or permission. Preserve source IDs, versions and access metadata. Apply authorization to the exact resources returned, and measure retrieval against known supporting evidence rather than only inspecting a few plausible neighbors.

15. Build RAG from a question and two documents

The concept: retrieve information before generating an answer

Retrieval-augmented generation, or RAG, supplies retrieved evidence as model input when answering a question. Retrieval changes what information is available during inference. It does not normally update the generator's weights.

The problem has two parts: find relevant evidence, then use it faithfully. A correct generator cannot reliably quote a source passage it never received. A good retriever cannot prevent the generator from misreading the retrieved passage. Evaluate these parts separately.

Derive the pipeline from the information dependencies

Before search, sources must be readable and identifiable. Parse them while preserving document structure and provenance. Split them into retrievable units called **chunks**, encode/index those units, and store source IDs, versions and access information.

At request time, identify the user's question and authorized corpus, retrieve candidates, optionally rerank them, select evidence that fits the context budget, and ask the generator to answer from it. Then check whether the answer's claims are supported. A citation needs a real source location supporting the nearby claim; its mere presence does not prove faithfulness.

The chunking rule and why it has this form

For fixed-length chunks of s units with overlap o:

$$\text{stride} = s - o, \qquad \text{start}_i = i(s - o), \qquad 0 \le o < s$$

s is chunk size, o the repeated region, and i a zero-based chunk index. After taking s units, move forward only $s - o$ so the next chunk repeats the final o units. The condition ensures positive progress. If overlap equals size, the stride becomes zero.

This is an algorithm we choose. The units might be words, characters or tokenizer tokens; those are not interchangeable. Larger chunks retain more surrounding context but can dilute a match and consume more prompt space. Overlap can preserve a boundary-spanning statement but creates duplication. Headings, tables and linked exceptions often need structure-aware handling rather than a fixed window alone.

What retrieval quality means

Choose a returned-result count k. For questions with known relevant units, recall@k is the fraction of relevant units that appear in those returned results. The denominator is all labeled relevant units, not k. Precision@k asks what fraction of returned results are relevant. Define relevance at the document,

chunk or evidence level consistently.

Then score whether the final answer is correct, supported and appropriately uncertain. A useful no-evidence answer may be a clarification or abstention rather than a fabricated fact. Increasing k can improve coverage while adding distractions and cost.

Worked example: observe where the exception goes

```
words = "refunds allowed within thirty days except customized products".split()
size, overlap = 5, 2
step = size - overlap
chunks = [" ".join(words[i:i+size])
          for i in range(0, len(words), step)]
print(chunks)
assert 0 <= overlap < size
```

Here the units are whitespace-separated words. Size five and overlap two give stride three. The first chunk ends at “days”; the second includes “thirty days except customized products.” Overlap carries context into the exception chunk. A third short tail is also produced because the simple loop keeps every start below the text length.

The lab only splits text. It performs no embedding, retrieval or generation, and it does not establish an optimal chunk size. Move the overlap assertion before calculating the stride if turning this into a function accepting user input.

Check your understanding

Question: How do you diagnose an incorrect RAG answer?

Answer: Inspect the source, parsed text, retrieved candidates, final evidence context and answer in that order. The first missing or corrupted fact identifies a useful place to intervene. Query rewriting should preserve the user's meaning rather than silently changing the question to something easier to answer.

16. Hybrid ranking and knowledge graphs without the hype

The concept: different searches preserve different signals

Lexical search compares words or terms. Dense retrieval compares learned vectors. Exact names, identifiers and rare phrases can favor lexical matching; paraphrases can favor dense retrieval. **Hybrid search** combines their candidates rather than assuming either signal always wins.

Their raw scores can have different scales and meanings. Adding those raw scores with equal weights is therefore not automatically meaningful. Rank fusion instead combines relative positions in result lists.

Derive reciprocal rank fusion

We want a contribution that is positive, decreases for lower-ranked items and can accumulate when multiple lists return the same document. One chosen rule is reciprocal rank fusion, RRF:

$$S(d) = \sum_{j:\, d \in L_j} \frac{1}{c + r_j(d)}$$

d is a document, L_j is retrieval list j, $r_j(d)$ is its one-based rank in that list, and $c > 0$ is a chosen offset. $S(d)$ is the fused score. A missing document contributes nothing from that list; do not invent a zero rank for it.

Why these terms? The reciprocal makes earlier ranks contribute more. The offset moderates the advantage of a single first-place appearance. Summation rewards appearing in multiple lists. This is a useful ranking design, not a theorem that its ordering must match relevance. The offset is not the number of documents returned. See the original RRF paper.

Graphs supply a different kind of structure

A **knowledge graph** stores entities and explicit relationships, often represented as subject–relation–object triples. Traversal follows those relationships to find connected evidence. Unlike vector proximity, an edge has a declared relationship type, though its extraction can still be wrong.

Graph construction needs entity identification, relationship extraction, source provenance, updates and permission enforcement. Use it when questions depend on relationships that are hard to recover through ordinary retrieval and the measured benefit justifies those costs. Graph traversal is not automatically reasoning correctness.

Worked example: combine positions, not raw similarities

```
lists = [["A", "B", "C"], ["B", "D", "A"]]
k0 = 60
scores = {}
for ranking in lists:
    for rank, doc in enumerate(ranking, start=1):
        scores[doc] = scores.get(doc, 0) + 1 / (k0 + rank)
print(sorted(scores, key=scores.get, reverse=True))
# B, A, D, C
```

`A` ranks first and third, so it gets `1/61 + 1/63`. `B` ranks second and first, so it gets `1/62 + 1/61`, slightly larger. `D` appears only at second place in one list and gets `1/62`. The final order is `B, A, D, C`.

The code uses `c = 60`, named `k0` in the lab. It learns no parameters and uses no document text. It demonstrates score fusion only; evaluate the final ordering against relevance labels on your own task.

Check your understanding

Question: Why might a graph be unnecessary even if your documents mention related people and projects?

Answer: Ordinary search may already retrieve enough supporting evidence. A graph adds another derived representation that can be incomplete, stale or incorrect. Compare a simple baseline with the additional graph on representative relationship questions before adopting the maintenance burden.

17. Tools and human review: from a suggestion to an action

The concept: a model output describes an operation

A **tool call** names an operation and provides arguments. Generating that data does not grant permission or perform the operation. An **executor** is the application component that validates the request and calls an allowed service.

A reliable tool integration needs a contract describing argument types, required fields, units, permissions, side effects and result states. These requirements come from the service and application, not from the model's confidence.

Derive the execution conditions

An action should be eligible only when all required conditions hold:

$$\text{eligible} = \text{known} \wedge \text{valid} \wedge \text{authorized} \wedge \text{approved}$$

$\wedge$ means logical AND: every condition must be true. `known` identifies an allowed tool, `valid` means its arguments meet the contract, `authorized` means current resource permissions allow it, and `approved` means any required user approval covers this exact action. If approval is not required by policy, that condition is satisfied without a prompt.

This is a logical specification, not a training objective. Eligibility allows an attempt; it does not guarantee completion. A network timeout, conflict or service failure can still prevent success.

Why approval needs concrete arguments

Approval of an action must be tied to the action's destination, content and relevant resource version. If those change after review, the old approval may no longer cover the operation. Otherwise a benign reviewed proposal could turn into a different action at execution time.

Separate outcome states such as denied, failed, succeeded and unknown. “Unknown” matters when the service might have completed an action but the caller did not receive its response. Treating every timeout as failure can cause duplicate actions when retrying.

Derive safe retry behavior

Give one intended operation a stable identity. When supported by the service, an **idempotency key** makes repeated requests for that same operation return its existing result rather than repeat its effect. Reusing the same key for different arguments should be rejected. If the service lacks that contract, reconciliation may require querying for the actual outcome or asking for review.

A local dictionary alone cannot atomically cover a remote side effect and a local receipt write. The gap between them is the crash problem explored next.

Worked example: inspect the checks before the mock result

```
def execute(proposal, confirmed, permissions):
    if proposal.get("tool") != "reminders.propose":
        return "unknown tool"
    if not proposal.get("text"):
        return "missing reminder text"
    if "reminders.write" not in permissions:
        return "permission required"
    if not confirmed:
        return "confirmation required"
    return "mock reminder accepted"

p = {"tool": "reminders.propose", "text": "Call Maya"}
print(execute(p, False, {"reminders.write"}))
print(execute(p, True, {"reminders.write"}))
```

The first call returns “confirmation required”; the second returns “mock reminder accepted.” No reminder API is called. The function checks a permitted name, nonempty text, a permission string and a confirmation flag. It deliberately omits real date handling, identity verification, durable receipts and network behavior.

A production contract would also validate text types, lengths, resolved dates and exact resource access. A boolean field saying “confirmed” is not sufficient evidence of user approval unless the application established it through a trusted path.

Check your understanding

Question: Why should the executor check permissions when the model was already told the allowed tools?

Answer: Instructions influence predictions; they do not enforce authority. The executor controls the actual service boundary and must reject unauthorized requests regardless of how they were generated.

18. Agents, memory and surviving a crash

The concept: an agent is a decision loop with state

An **agent** repeatedly observes state, chooses a next step, receives results and decides whether to continue. It needs explicit goals, allowed operations, stopping conditions and resource budgets. Merely calling a model several times does not establish useful autonomy.

A workflow has predefined transitions; an agent can choose among allowed transitions based on observations. In practice, reliable systems often combine fixed workflow structure with a few model-driven decisions.

The state-transition rule

A general description is:

$$s_{t+1} = F(s_t, a_t, r_t)$$

s_t is current state at step t, a_t the attempted action, r_t its observed result, and F the application's transition rule. The next state records what changed and what may happen next. This notation is an organizing model, not a universal agent implementation.

Derive the needed state from recovery requirements. To resume, the application must know the goal, completed stages, pending operations, available evidence, budget spent and relevant operation identities. Saving only the latest natural-language response usually loses important distinctions.

Why memory has several meanings

Conversation history holds recent messages. Working state holds intermediate task data. Retrieved memory supplies records from storage. Durable checkpoints save resumable execution state. Model weights encode learned statistical behavior. A KV cache holds reusable runtime tensors. These differ in update procedure, lifetime and purpose.

Trimming conversation history does not erase database records. Saving a checkpoint does not train the weights. Retrieving a note supplies information but does not grant permission to act on it.

Derive the crash boundary

Suppose a worker asks a remote service to create a reminder, the service succeeds, and the worker crashes before saving the receipt. On restart, the checkpoint still says the action is pending. Retrying blindly may create a duplicate. Saving “done” before calling the service has the opposite failure: a crash can leave a false success record.

Therefore saving local state alone cannot guarantee exactly one remote effect. Use service-supported idempotency, a transaction covering the relevant boundary when possible, or reconciliation to determine whether the action occurred. Explicitly retain unknown outcomes when evidence is insufficient.

Worked example: a bounded decision loop

```
budget = 3
results = [[], [], ["supporting passage"]]
state = "searching"
for attempt in range(budget):
    evidence = results[attempt]
    if evidence:
        state = "ready to answer"
        break
else:
    state = "insufficient evidence"
print(state)
```

The loop allows at most three attempts. Empty evidence keeps it searching; nonempty evidence sets the state to “ready to answer” and breaks. The `else` attached to the `for` runs only if the loop exhausts without a break, marking insufficient evidence.

The results are supplied in advance. No live search or durable storage occurs, so this illustrates stopping behavior rather than complete crash recovery. To add recovery, persist attempts, evidence and pending operation identities at defined boundaries.

Check your understanding

Question: What prevents an agent from searching forever when evidence is unavailable?

Answer: Explicit stopping rules: attempt limits, elapsed time, cost budget, repeated-result detection and a policy for insufficient evidence. Evaluate successful completion, unnecessary actions, recovery correctness and budget use together. More steps are useful only if they improve the intended result.

19. Safety and prompt injection in plain language

The concept: information does not automatically carry authority

A model may read instructions, user requests, retrieved documents and tool results in the same context. These sources have different trust levels. **Prompt injection** occurs when lower-trust content tries to redirect behavior beyond its authority, for example by telling the assistant to ignore its task and export private information.

The core problem is a trust boundary: text that should be treated as evidence is interpreted as an instruction controlling the application. An attack can be phrased politely and need not contain any particular suspicious word.

Derive the enforcement rule

Authority to execute an operation must come from a trusted source: authenticated identity, current permissions and the application's execution policy. Retrieved content cannot grant new capabilities simply by requesting them.

A useful invariant is: **every attempted tool action must remain within the current user's permitted resources and the application's allowed operations**. An invariant is a condition the system maintains across all execution paths. No special numerical formula is needed; this is a requirement on program behavior.

From that requirement follow concrete controls. Limit which tools an executor exposes. Check arguments and exact resource access. Restrict destinations and available credentials. Apply isolation and network restrictions appropriate to the operation. Keep these checks effective even when the model proposes a malicious action.

Why prompt wording alone is insufficient

Instructions can help the model distinguish source text from commands, but they do not control the external API. A model might still follow hostile content or misinterpret a benign sentence. The component holding credentials must enforce the boundary independently.

An allowlist of tool names is only one layer. A permitted `notes.read` tool could still leak information if it accepts another user's note ID. Check scope within permitted tools as well. Search filters, caches and logs must also respect access; protecting only the final answer leaves other exposure paths.

Worked example: enforce a capability boundary

```
allowed = {"notes.search", "notes.read"}
proposal = {"tool": "notes.export_all", "destination": "outside"}
if proposal["tool"] not in allowed:
    result = "blocked by executor policy"
else:
    result = "continue with argument and access checks"
print(result)
```

The proposed `notes.export_all` operation is absent from the allowed set, so ordinary code blocks it. The outcome does not depend on the model accepting an instruction to behave safely.

If the proposal named `notes.read`, the lab would only say to continue with argument and access checks. It would not establish authorization. This distinction is why the second branch does not directly execute the tool.

Derive useful adversarial tests

Choose an asset to protect, an attacker-controlled surface and a forbidden result. Then place hostile instructions in that surface and observe whether the protected operation occurs. Also test legitimate requests: a system that blocks everything is secure against that operation but useless for its intended task.

Preserve source information so failures can be traced to a document or tool result. Avoid storing private content unnecessarily in prompts, traces or diagnostics; data copied into a new system creates another access boundary.

Check your understanding

Question: A retrieved document says the user approved exporting all notes. Does that establish approval?

Answer: No. It is a claim inside retrieved content. Approval and permissions must be established through a trusted application path and tied to the specific operation. The executor's checks should hold even if the model repeats the claim confidently.

20. MCP and A2A: understand the interface before the acronym

The concept: components need a shared communication contract

A **protocol** defines how components exchange messages and interpret their outcomes. It can specify message structures, discovery, lifecycle and error behavior. It does not determine whether the underlying model is intelligent or whether a requested action is permitted.

MCP, Model Context Protocol, provides conventions for connecting model applications to capabilities such as tools and resources. **A2A**, Agent2Agent, provides conventions for communicating with agents and their task-oriented capabilities. The exact features and messages depend on the protocol version and implementation.

Derive what a caller needs to know

Before calling another component, a client needs to establish:

1. What capability is available and what it means.
2. What arguments and types it accepts.
3. Who the caller is and what that identity may do.
4. How the operation's result, errors and completion state are represented.
5. What to do about cancellation, timeouts and retries.

Without discovery or configuration, the caller does not know which operation exists. Without a schema, it cannot construct an interpretable request. Without authorization, syntactic validity could expose forbidden work. Without result semantics, it might mistake acceptance for completion.

These dependencies explain the contract. Memorizing a message envelope without understanding them will not make an integration reliable.

Separate four layers

Transport carries messages, such as an HTTP connection. **Protocol** defines interaction conventions and message semantics. **Authentication** establishes identity. **Authorization** decides whether that identity may perform an operation on a resource. One layer does not automatically supply all the others.

There is no universal mathematical formula for protocol correctness. Use an explicit state machine and contract: a valid transition requires the expected message, compatible version, authenticated/authorized context and appropriate task state. A successful response must be interpreted according to the service's documented semantics.

Versions matter because capabilities and required fields can evolve. For a real integration, consult the negotiated version's MCP specification or A2A specification. This lesson intentionally teaches stable concepts rather than claiming one hand-written dictionary is a compliant protocol message.

Worked example: validate a protocol-neutral request

```
supported = {"notes.search": {"query"}}
request = {"tool": "notes.search", "arguments": {"query": "Atlas"}}
name = request["tool"]
required = supported.get(name)
valid = required is not None and required <= request["arguments"].keys()
print(valid)  # True
```

`supported` maps a tool name to required argument names. The subset test checks that every required key exists. The result is `True` because `query` is present.

The lab does not check value types, extra fields, identity, permissions or protocol envelopes, and it sends nothing over a network. It demonstrates one contract check. A real implementation would use the actual SDK/schema and version-specific lifecycle rather than wrapping this dictionary and assuming compatibility.

Check your understanding

Question: If a request is protocol-valid, can we execute it immediately?

Answer: Only after the application's other requirements are satisfied. A well-formed request can still use expired credentials, target the wrong service, exceed resource scope or describe an unauthorized action. Interface compatibility and authority answer different questions.

21. Evaluation and statistics you can actually use

The concept: a metric is a precise question about outcomes

“Good” is not a metric until we define the unit being scored, the correct outcome and the denominator. A unit might be a token, classification, whole request or retrieved document. Changing that unit changes the question.

First define what counts as success, including partial success, clarification and unsupported answers. Then choose a representative test set and a scoring procedure. A model judge also needs validation against trusted judgments; its output is another measurement with possible error.

Derive classification metrics from the counts

For a chosen positive class, **TP** is correctly predicted positive, **FP** incorrectly predicted positive, **FN** missed positive, and **TN** correctly predicted negative.

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}$$

Accuracy asks what fraction of all predictions are correct. Precision P asks what fraction of predicted positives are correct, so its denominator includes FP. Recall R asks what fraction of actual positives were found, so its denominator includes FN.

F_1 is the harmonic mean of precision and recall: taking the reciprocal of their average reciprocals gives `2PR/(P+R)`. It penalizes a low value in either component. It is a chosen summary, not a substitute for deciding which errors cost more. Specify behavior for zero denominators; do not silently divide by zero.

For retrieval, precision@k uses relevant returned items divided by the number returned, usually k. Recall@k uses relevant returned items divided by all labeled relevant items. Define the evidence unit consistently and handle no-relevant-evidence questions separately.

Why sample size changes confidence in the result

An observed fraction $\hat{p}$ is an estimate, not an exact property of future behavior. Under independent comparable Bernoulli trials, a success indicator has variance $p(1-p)$. Averaging n independent indicators divides that variance by n. Taking the square root and substituting the observed fraction gives an approximate standard error:

$$SE \approx \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

n counts independent trials. This is not itself a universal confidence interval, and tiny samples or proportions near zero/one need more careful methods. Duplicated examples violate the independence assumption and can overstate certainty.

Worked example: keep the failed case visible

```
expected = ["reminder", "clarify", "note", "cancel"]
observed = ["reminder", "reminder", "note", "cancel"]
correct = [a == b for a, b in zip(expected, observed)]
print(sum(correct) / len(correct))  # 0.75
for i, ok in enumerate(correct):
    if not ok:
        print("Inspect case", i, expected[i], observed[i])
```

Three out of four predicted labels match, giving accuracy `0.75`. The incorrect case turns a needed clarification into a reminder. That mistake may be more consequential than a harmless label mismatch, which is why we keep case-level failures alongside the average.

Class imbalance can hide rare failures: predicting the common class may achieve high accuracy while missing every rare positive. Inspect class metrics and important slices such as language, ambiguity, permission status and input length.

Check your understanding

Question: Can two systems have equal accuracy but different usefulness?

Answer: Yes. They may fail on different classes or produce errors with different costs. Offline labeled evaluation compares controlled cases; online observation measures behavior under actual traffic and changing conditions. Use both, with versioned inputs and a clear scoring policy.

22. Serving, latency, cost and model routing

The concept: user waiting time spans the whole system

Latency measures elapsed time for a request. **Throughput** measures completed work per unit time. **Concurrency** counts work in flight. They describe different properties, so a change can improve one and worsen another.

For an AI response, waiting can include admission, retrieval, prompt processing and generation. **Time to first token** includes everything before the first generated token appears. **Inter-token latency** measures subsequent token spacing. Report the boundary and workload, not just “speed.”

Derive a latency budget and average concurrency

For sequential stages, total duration is the sum of stage durations. Overlapping stages require measuring the critical path rather than summing overlapping intervals twice.

Under stable long-run conditions, Little's law relates averages:

$$N = \lambda W$$

N is average in-flight work, λ completed throughput in requests per second, and W average time in seconds within the **same** system boundary. Intuitively, each arrival occupies the system for some time; throughput times average occupancy duration gives average occupied requests. This relation is accounting under its assumptions, not proof the system can sustain additional demand.

Queueing grows when demand approaches capacity. Batching may improve hardware utilization and aggregate throughput while increasing individual wait. Measure representative prompt/output lengths and tail latency as well as averages.

Derive direct routing versus fallback cost

Suppose a fraction p of requests needs a large model, with small-model cost C_s and large-model cost C_l. A perfect direct route uses exactly one model per request:

$$C_{direct} = (1 - p)C_s + pC_l$$

If every request tries the small model first and fraction p then falls back, the expected model-call cost is:

$$C_{fallback} = C_s + pC_l$$

The difference is pC_s, because escalated requests paid for both models. These equations omit router overhead and assume fixed per-call costs and the same escalation fraction. Actual costs depend on tokens, retries, runtime and routing accuracy.

What memory optimizations change

Quantization stores values with fewer bits, reducing representation size but potentially changing quality. For P parameters stored at b bits each, raw weight storage is approximately $Pb/8$ bytes; scales and runtime overhead add more. KV caching reuses computation but consumes memory. Response caching reuses an answer and therefore also needs freshness, identity and access rules.

Worked example: count every attempted call

```
small_cost, large_cost = 1.0, 5.0
fraction_large = 0.30
direct = ((1-fraction_large) * small_cost
          + fraction_large * large_cost)
fallback = small_cost + fraction_large * large_cost
print(direct, fallback)  # 2.2, 2.5
```

Costs `1` and `5`, with `30%` needing the larger model, give direct cost `2.2` and fallback cost `2.5` in arbitrary units. These are calculated expectations, not measured currency prices or evidence of a real router's accuracy.

Check your understanding

Question: How do you evaluate a faster serving configuration?

Answer: Hold request mix and scoring policy comparable. Measure task quality, first-token time, output speed, throughput, tail latency, memory and failure rate. A routing change that saves compute but sends hard cases to an inadequate model may fail the user's actual objective.

23. LLMOps, architecture and data engineering

The concept: the deployed prediction depends on many artifacts

LLMOps is the operational work needed to reproduce, evaluate, release and observe an AI system. A response depends on more than model weights: tokenizer, prompt, data, retrieval index, permissions, runtime settings and application code all affect behavior.

A **version** identifies an artifact state. A **trace** records events along one request. A **release gate** states measurable conditions a candidate must satisfy before broader use. These solve reproduction, diagnosis and rollout problems respectively.

Derive what must be recorded

To reproduce a failure, you need the inputs and the transformations that produced the output. Therefore retain relevant input/source versions, model and adapter identities, tokenizer/template, prompt, retrieval settings, index version, runtime configuration and scoring policy. Use protected identifiers and appropriate redaction where raw content is sensitive.

The principle is causal: if an unrecorded component could change the output, reproducing only the weights may not reproduce the behavior. Changing the embedding model while keeping an old index can break the representation contract without causing a code exception.

The release rule and why averages are insufficient

A release decision can be expressed as a conjunction of required conditions: acceptable task quality, acceptable risk on protected cases, latency within budget and a working recovery path. Thresholds are product decisions supported by measurement, not universal constants.

An aggregate quality score can hide regressions in a rare critical subset. Compare candidate and baseline on the same held-out cases and inspect failure categories. A canary release exposes a limited amount of representative traffic; rollback restores the previous known artifact set if release criteria fail. A rollback plan must include dependent data/index compatibility.

Derive reliable data-pipeline behavior

Derived records need source IDs, versions and ownership. A retried job should not create accidental duplicates. A slow old extraction should not overwrite a newer version. A deleted source should trigger the defined removal or retention policy for derived chunks, embeddings and caches.

These requirements imply stable identities, explicit version checks, idempotent processing where possible and observable failure states. More infrastructure does not remove the need to define those semantics.

Worked example: successful stages can end in failure

```
trace = [
    {"stage": "retrieve", "ms": 20, "ok": True},
    {"stage": "generate", "ms": 120, "ok": True},
    {"stage": "validate", "ms": 2, "ok": False},
]
print("Total:", sum(x["ms"] for x in trace), "ms")
print("First failed stage:", next(x["stage"] for x in trace if not x["ok"]))
```

Sequential stage durations add to `142 ms`. Retrieval and generation report success, but validation fails. The lab prints `validate` as the first failed stage. It does not demonstrate a working production trace collector; it demonstrates the information a minimal trace makes visible.

This total is valid because the supplied stages are sequential. In a real trace, concurrent spans can overlap, so summing every span duration can overcount elapsed time. A successful HTTP request also need not mean a correct answer.

Check your understanding

Question: What should a useful production dashboard show besides errors and uptime?

Answer: Quality signals tied to task outcomes, latency distributions, resource use, data/version changes and failure categories. Include enough trace context to investigate a specific request, while controlling access to sensitive data. Monitoring detects change; an investigation must still establish its cause.

24. Fine-tuning, LoRA and QLoRA through your own adapter

The concept: adaptation changes parameters to change behavior

Pretraining learns broad patterns from a large training objective. **Fine-tuning** continues optimization for a particular dataset or task. It is useful when the needed behavior is better learned through examples than supplied through instructions or retrieved facts.

Start by identifying the failure. Missing up-to-date facts often suggest retrieval; inconsistent output style or a repeatable task behavior may suggest adaptation. Compare against a prompting baseline and use independent evaluation before claiming improvement.

Derive LoRA from a full matrix update

A linear map in column-vector notation is $y = Wx$, where W has shape `[O, I]`, x has length I, and y length O. Full fine-tuning can change every entry of W. **LoRA** instead freezes W and learns a factorized adjustment:

$$W' = W + sBA, \qquad y = Wx + sB(Ax)$$

A has shape `[r, I]`, B has shape `[O, r]`, r is the chosen rank and s a scale. `A @ x` first maps the input to r coordinates; `B` maps it back to output width. Their product has the same shape as W, so it can be added.

The adjustment's rank is at most r. This constrains the update rather than representing every possible full update at small rank. Training learns A and B while the base usually stays frozen.

Derive the parameter saving and scale

The full matrix contains OI parameters. The factors contain $rI + Or = r(I + O)$. When r is much smaller than both widths, fewer values need gradients and optimizer state. The base still occupies memory, so this ratio is not the total training-memory ratio.

A common standard scale is $s = \alpha / r$; rank-stabilized LoRA uses $s = \alpha / \sqrt{r}$. α and r are chosen settings. Neither scale is the universal definition of every adapter. Check the actual method; PEFT's LoRA reference documents its rank-stabilized option.

QLoRA additionally uses a quantized frozen base during adaptation, with adapter parameters trained in an appropriate computation setup. Deployment quantization is a separate decision about the served artifact. Quantization can affect outputs, so evaluate after conversion or merging.

Worked example: count parameters before comparing quality

```
import math
input_width = output_width = 1024
rank, alpha = 32, 64
full = input_width * output_width
adapter = rank * (input_width + output_width)
print(full, adapter)  # 1048576 65536
print("standard scale:", alpha / rank)       # 2.0
print("rsLoRA scale:", alpha / math.sqrt(rank))  # about 11.31
```

A `1024 × 1024` matrix has `1,048,576` entries. Rank-32 factors contain `32*(1024+1024) = 65,536`, one sixteenth as many. With alpha 64, standard scale is `2`; rank-stabilized scale is about `11.31`. These are scale calculations, not evidence that one setting must perform better.

The earlier NoteEchoes configuration review recorded rank 32, alpha 64, dropout 0.05 and `use_rslora=true`. That is a dated project snapshot, not a check of the currently installed model. The distinction matters because interpreting its scale as standard LoRA would describe a different computation.

Check your understanding

Question: What should you check when adaptation fails?

Answer: Inspect data quality, target formatting, scored-token masks, truncation, optimizer parameter selection and learning rate before assuming more rank is needed. Then evaluate the actual deployment artifact on held-out cases, including regressions the tuning objective may not capture.

25. Preferences, reinforcement learning and distillation

The concept: different feedback supplies different information

Training needs a signal that makes some behavior preferable to other behavior. A correct answer, a pairwise preference, a numerical reward and a teacher's probability distribution supply different kinds of information. The objective determines how that information affects parameters.

Supervised fine-tuning increases the probability of supplied target outputs. **Preference optimization** uses judgments comparing outputs. **Reinforcement learning** optimizes a policy using rewards from sampled behavior. **Distillation** trains a student to match selected behavior or distributions from a teacher. These categories overlap in practice but are not interchangeable names for the same loss.

Derive the reward objective

A language-model policy $\pi_\theta(y \mid x)$ assigns probabilities to responses y given input x. Parameters are θ. If $R(x, y)$ scores a response, the expected reward for a fixed input is:

$$J(\theta) = \sum_y \pi_\theta(y \mid x) R(x, y) = \mathbb{E}_{y \sim \pi_\theta}[R(x, y)]$$

$\sum_y$ adds over possible responses; $\mathbb{E}$ means an expectation, or probability-weighted average. We weight a response's reward by how often the current policy produces it. Improving a rare response helps little unless its probability also changes. Real training also averages across inputs and uses approximations rather than enumerating every possible response.

This objective is maximized, unlike a loss that is minimized. It describes the goal, not a complete learning algorithm. Practical methods define gradient estimators, sampling procedures and constraints. Pairwise preference methods can learn from comparisons without directly using this exact scalar-reward setup.

Why constraints and reward design matter

A reward measures a proxy for the intended outcome. Optimizing that proxy can exploit shortcuts. A reference-policy penalty can discourage excessively large behavior shifts, but it does not prove the reward is correct. Evaluate usefulness and undesirable changes independently.

In distillation, one possible objective is cross-entropy from a teacher distribution q to a student distribution p:

$$L_{distill} = -\sum_i q_i \ln p_i$$

i indexes token candidates; q_i is the teacher's target probability and p_i the student's probability under comparable context. This generalizes the one-correct-token loss from Lesson 7. Sequence-based distillation instead supplies teacher-generated target text. The student can inherit errors or coverage gaps from the teacher and dataset.

Worked example: a measurable reward can be wrong

```
answers = ["Source: a real supporting passage", "source source source"]
def bad_reward(text):
    return text.lower().count("source")
for text in answers:
    print(bad_reward(text), text)
```

The first answer contains one occurrence of “source”; the second contains three. The reward therefore prefers repetition even though it supplies no evidence. The function is calculated correctly; the objective fails to represent the intended behavior.

A better evaluation would inspect whether citations identify genuine supporting passages and whether the answer addresses the question. That still requires defining and validating a scoring procedure rather than adding a new keyword.

Check your understanding

Question: Is giving a response more inference time the same as reinforcement learning?

Answer: No. Additional inference computation can generate or evaluate more intermediate work using current parameters. Learning changes parameters through an objective. User feedback also does not automatically train a model unless a learning process actually consumes it.

26. Multimodal AI and the complete NoteEchoes journey

The concept: each representation change can lose information

Multimodal AI processes more than one kind of information, such as text, audio or images. A system might use one joint model or several specialized components. In either case, representation boundaries deserve explicit inputs, outputs and evaluation.

For a voice application, the main chain can be audio waveform → transcript → structured intent → validated action → execution receipt. A later stage can behave correctly relative to its input while the end-to-end outcome remains wrong because an earlier stage corrupted information.

Derive the diagnostic method

Start with the user's intended outcome and retain observable artifacts at each boundary. Compare audio with transcript, transcript with extracted fields, fields with validated arguments and arguments with execution result. The first incorrect boundary narrows the investigation.

Why not evaluate only the final label? That label hides which transformation lost the information. Why not evaluate only transcription? A perfect transcript can still yield the wrong date or an unauthorized action. Stage metrics and end-to-end success answer different questions.

Derive word error rate

For a reference transcript containing N words, align a hypothesis to it using word substitutions S, deletions D and insertions I. Word error rate is:

$$\mathrm{WER} = \frac{S + D + I}{N}$$

The numerator counts edits; the denominator counts reference words. Multiplying by 100 expresses a percentage. Insertions can make WER exceed one, so it is not automatically bounded at 100%. Define a separate policy when the reference has zero words.

The alignment and text-normalization rules affect these counts, so keep them consistent. WER treats word edits uniformly; an incorrect name or date can matter much more to the action than a missed filler word. Measure important entities and end-to-end task outcomes too.

What differs across modalities?

Audio needs sample rate, channel, timing and encoding contracts. Images need dimensions, orientation and any resizing/cropping rules. Video adds timing and frame selection. Text produced by OCR or speech recognition is a derived interpretation, not identical to the original evidence.

Track provenance across those transformations. Preserve enough information to investigate errors within the application's privacy and retention rules. Model capability labels alone do not establish measured performance on the languages, devices or input conditions you need.

Worked example: identify the artifacts before calling models

```
stages = {
    "audio": "recorded waveform",
    "transcript": "Remind me to call Maya tomorrow at 6 pm",
    "intent": "reminder",
    "tool_proposal": "reminders.propose",
    "execution": "not attempted in this teaching example",
}
for stage, value in stages.items():
    print(stage + ":", value)
```

The dictionary names the stage artifacts for a fictional reminder. Printing the transcript or intent does not perform speech recognition or intent inference. The execution field explicitly records that no action was attempted.

To turn this into a real investigation, collect the actual stage outputs for controlled requests. If audio says Tuesday and the transcript says Thursday, the error precedes intent extraction. If the transcript is right but the proposed date is wrong, investigate interpretation and date resolution.

Check your understanding

Question: Can a lower WER coexist with worse reminder accuracy?

Answer: Yes. An improvement on many low-impact words can outweigh more mistakes on critical dates or names in the WER average. Define the user's success condition and inspect important subsets. For NoteEchoes claims, distinguish measured reports from illustrative examples and dated configuration snapshots.

27. Coding, SQL, system design and behavioral interviews

The concept: explain a solution through its requirements

An interview tests whether you can turn an ambiguous request into a correct, defensible solution. Coding, SQL, design and behavioral questions use different artifacts, but all benefit from explicit assumptions, a clear mechanism and evidence.

Begin by defining input, output, constraints and success. Then choose an approach because it satisfies those requirements. Naming a data structure, framework or cloud service without showing its role leaves the reasoning incomplete.

Derive a coding approach from the operation

Suppose a task asks for the largest k scores among n items. Sorting all items is simple and typically costs $O(n \log n)$ comparisons. If k is small, a min-heap holding the best k seen so far can avoid fully sorting the rest.

The heap's invariant is: after processing a prefix of the input, it contains the best k items from that prefix, or all items if fewer than k have been seen. Its smallest retained score is at the root. When a new item beats that root, replace it; otherwise discard it. Each update costs $O(\log k)$, yielding $O(n \log k)$ work and $O(k)$ retained storage, plus ordering the final results if required.

Here $O(\cdot)$ describes how resource use grows with input size, not an exact duration. For $k = 1$, a simple linear maximum scan suffices. Algorithm choice depends on constraints; a library function may choose a different strategy internally.

Derive SQL correctness from row meaning

Before writing a join, state what one row represents. A one-to-many join repeats the “one” side for each matching row. Summing a value from that side afterward can double count it. To prevent this, aggregate at the intended unit or join to a correctly grouped subquery.

Transactions group operations whose consistency must be preserved together. Indexes accelerate selected access patterns at a storage/write cost. Neither a transaction nor an index fixes an undefined data model. Explain the exact query and consistency requirement before proposing either.

Derive a system design answer

Requirements determine the request/data flow. Workload estimates identify possible bottlenecks. Failure requirements determine retries, idempotency, persistence and recovery. Finally, metrics establish whether the design meets its purpose. This is a chain of decisions, not a checklist of products.

For behavioral answers, explain the situation, your responsibility, the action you chose and the supported outcome. Separate your contribution from team work. If a result lacks a baseline or denominator, qualify the claim rather than inventing one.

Worked example: ask a library for the top results

```
import heapq
scores = [(0.8, "note-A"), (0.3, "note-B"), (0.9, "note-C")]
print(heapq.nlargest(2, scores))
# [(0.9, 'note-C'), (0.8, 'note-A')]
```

`nlargest(2, scores)` returns the two highest tuples, led by score. If scores tie, tuple comparison can use the note ID as a secondary ordering, so define tie behavior explicitly when it matters. The lab demonstrates the result contract; it does not implement or benchmark the heap algorithm described above.

Check your understanding

Question: What makes an answer defensible when you forget a memorized solution?

Answer: State a correct baseline, identify its cost, derive an improvement from the constraint and test edge cases. For project questions, connect the claimed result to a baseline, metric, dataset and actual contribution. Sound reasoning and observable evidence transfer better than memorized terminology.

28. Your capstone and a learning plan that allows understanding

The concept: demonstrate one complete, measurable behavior

A **capstone** is a small project that connects the course concepts into a working system. Its purpose is to demonstrate understanding through inspectable behavior. A narrow end-to-end task usually provides better evidence than many disconnected features.

Start with a user outcome, expected inputs, allowed data and a completion condition. A baseline establishes what the simplest approach can do. Add retrieval, a model or an agent only when the requirement or measured failure justifies it.

Derive the acceptance contract

Define test cases and decide what each metric scores. Useful separate questions include: does the output match the schema, does it identify the correct intent, does its answer have supporting evidence, and are attempted actions authorized?

For a metric m:

$$\text{pass_rate}_m = \frac{\text{passed_cases}_m}{\text{eligible_cases}_m}$$

m names the metric, and eligible cases form its denominator. Eligibility can differ: citation support may apply to answered factual questions while clarification accuracy applies to ambiguous inputs. Specify that distinction rather than comparing percentages from different populations as if they were the same.

If every stage must succeed for the whole request to pass, count whole-request success directly. You cannot generally multiply separately measured stage accuracies: failures can be correlated, and denominators can differ. Record case-level results so joint success is observable.

What artifacts make learning reviewable?

Keep a versioned configuration, representative inputs, expected outcomes, actual outputs, scoring policy, failure labels and instructions to reproduce a run. A demonstration should show a successful case, a failure and the recovery behavior. Include limitations rather than selecting only easy examples.

Choose release criteria before inspecting the final results. Criteria depend on the product: unsupported answers, unauthorized actions, poor completion or unacceptable latency may block readiness. There is no universal percentage that makes every AI system ready.

Worked example: separate success dimensions

```
results = {"schema": 15, "intent": 13, "safe_action": 15}
cases = 15
for metric, passed in results.items():
    print(metric, f"{passed}/{cases}", f"{passed/cases:.1%}")
print("Next step: inspect the two intent failures.")
```

All 15 synthetic cases pass schema and safe-action checks; only 13 pass intent checks, about `86.7%`. The printed summary exposes two intent failures even though the output shape looks perfect. These are supplied counts, not results from running an assistant.

The next useful step is to inspect those two cases and determine their cause. Adding more features before understanding them makes the system harder to diagnose. Also record whether any checks are vacuous: “no forbidden action” may be true simply because nothing was executed.

A learning route based on understanding

Use the 30 days as sessions, not proof of mastery on a deadline. For each lesson, first explain the mechanism without numbers, then derive or justify its rule, then calculate the example. Finally change one condition and predict the result before running it.

Check your understanding

Question: When is a lesson or capstone complete enough to move forward?

Answer: When you can explain its mechanism, reproduce the observed result and identify a meaningful limitation. Mark progress based on that evidence. Return to weak prerequisites when an advanced explanation feels like memorizing symbols.

29. Your parameter map and a gentle glossary

The concept: identify where a setting acts before changing it

The word “parameter” is often used loosely. A **learned parameter** is a value optimized during training, such as a weight. A **hyperparameter** configures the learning procedure. An **architecture choice** defines the computation's structure. An **inference setting** controls how current weights are used. A **retrieval setting** controls evidence search.

These categories can interact, but they affect different stages. Raising temperature cannot train a better representation. Increasing retrieval depth cannot correct an unauthorized executor. The first step is to locate the observed failure in the pipeline.

The rules behind common knobs

Learning rate multiplies an optimizer's gradient-based update; it is not the update itself. Batch size controls examples processed together. Context length limits positions; embedding width controls coordinates per representation. LoRA rank bounds the factorized adjustment's capacity. Temperature scales score differences. Retrieval k controls the number of returned candidates.

Each setting needs a defined unit and stage. A chunk size of 500 words differs from 500 tokenizer tokens. An output-token limit differs from total context capacity. Model weight precision can differ from KV-cache precision. Before changing a number, write down what it counts and what operation reads it.

Derive a controlled comparison

Let a baseline score be M_A and a trial score be M_B, measured using the same metric and comparable cases. The absolute change is:

$$\Delta M = M_B - M_A$$

For $M_A \neq 0$, relative change is $(M_B - M_A)/M_A$. If accuracy rises from `0.80` to `0.84`, the absolute gain is four percentage points; the relative gain is five percent. These answer different questions. For an error metric, a decrease may be the improvement, so define the direction explicitly.

A comparison is interpretable only if plausible competing causes are controlled. Keep data, prompt, model, scoring procedure and runtime conditions comparable unless one of those is the intended change. For stochastic runs, use repeated trials or a paired seed plan and report variability. One fixed seed improves reproducibility; it does not eliminate uncertainty.

From symptom to investigation

If training loss grows, inspect data, objective, gradients and optimizer settings. If exact search misses evidence, inspect representations and labels before tuning the approximate index. If schema validity is high but actions are wrong, inspect interpretation and validation. If first-token wait is high, measure queueing, retrieval and prefill before assuming decoding is slow.

These are diagnostic starting points, not guaranteed fixes. The observation tells you which hypothesis to test; the controlled experiment tells you whether the intervention helped.

Worked example: verify the intended configuration change

```
settings = {"learning_rate": 0.1, "steps": 5, "seed": 7}
trial = dict(settings)
trial["learning_rate"] = 0.01
changed = [key for key in settings if settings[key] != trial[key]]
print(changed)  # ['learning_rate']
assert len(changed) == 1
```

`dict(settings)` copies the baseline mapping. The trial changes only `learning_rate`, and the comparison reports that one key. The assertion confirms a single configuration difference. No model is trained, so this does not prove the new rate is better.

To complete the experiment, predict the effect, run comparable trials, collect the relevant trajectories and compare quality, latency and memory as appropriate. Retain or revert based on the defined objective.

Check your understanding

Question: Why can changing one setting still fail to establish its effect?

Answer: Other uncontrolled factors may differ: sampled data, random initialization, traffic, source versions or measurement policy. Record those conditions and repeat where necessary. A useful glossary connects each term to an operation you can inspect rather than a definition you can only repeat.

30. Optional workshop: read and run the complete tiny decoder

The concept: connect the calculation, the objective and the update

This workshop assembles the earlier ideas into a small causal language model. The **forward pass** maps token IDs to next-token logits. The **loss** scores the correct next tokens. The **backward pass** computes gradients. The **optimizer** changes parameters. **Generation** uses current parameters repeatedly to append tokens.

Read those interfaces first. The complete program is manageable when every operation has an input, output and purpose. It is a teaching decoder trained on repeated synthetic text, not a replica of the NoteEchoes model or evidence of broad language ability.

Derive the tensor route before reading the code

Let B be batch size, T sequence length, D hidden width, H head count and V vocabulary size. Integer IDs have shape `[B, T]`. Token and position lookup create `[B, T, D]` vectors. Each transformer block preserves that outer shape. The final vocabulary projection produces `[B, T, V]` logits.

Attention splits the feature axis into H heads of width $d_h = D/H$. This requires divisibility. Queries and keys have shape `[B, H, T, d_h]`; comparing each query to each key gives `[B, H, T, T]` scores. A triangular mask excludes future positions. Mixing values and joining heads restores `[B, T, D]`.

The block uses normalization, residual connections and a feed-forward network with GELU, a smooth nonlinear activation. These are chosen architectural operations. The projection and embedding coefficients are learned parameters. This decoder uses learned absolute positions and ordinary multi-head attention rather than rotary positions or grouped-query attention.

Derive the shifted targets and loss

If a training window contains IDs at positions `i` through `i+T-1`, the targets are positions `i+1` through `i+T`. Each position predicts its successor. The causal mask ensures its representation cannot read later input positions that would reveal that answer.

The loss flattens `[B, T, V]` into `[B*T, V]` and targets into `[B*T]`, then applies mean next-token cross-entropy. The last axis always remains the candidate vocabulary. Flattening is a rearrangement for scoring, not permission to mix token identities or alter temporal order.

`loss.backward()` applies the chain rule to the recorded computation. The program uses AdamW, whose update includes adaptive gradient statistics and weight decay, so it is more elaborate than Lesson 6's plain `w -= learning_rate * gradient`. The same dependency logic still supplies its gradients.

Derive the generation loop

Take logits at the final position, choose a next token, append its ID and repeat. The supplied code uses greedy `argmax` selection and crops input to its context length. It recomputes the cropped prefix each time: there is no KV-cache implementation here. Generated tokens do not update weights.

`model.eval()` selects evaluation behavior for relevant layers; `torch.no_grad()` prevents recording a gradient graph during generation. Neither line establishes that output quality is good.

Worked example: run the complete source

This optional lab requires Python and PyTorch. From the labs directory, run `python3 tiny_decoder.py --steps 40 --width 32 --heads 4`. Width 32 and four heads give eight coordinates per head. The default batch contains eight sampled windows. The seed supports reproducibility within comparable environments; exact floating-point outputs can vary across versions or devices.

```
"""A teaching decoder, not Qwen and not a production assistant.
Needs Python 3 and PyTorch. Uses only a small synthetic local corpus.
Example: python3 tiny_decoder.py --steps 40 --width 32 --heads 4
"""
import argparse
import math
import torch
from torch import nn
from torch.nn import functional as F

class Attention(nn.Module):
    def __init__(self, width, heads):
        super().__init__()
        self.heads = heads
        self.head_width = width // heads
        self.qkv = nn.Linear(width, 3 * width)
        self.output = nn.Linear(width, width)

    def forward(self, x):
        batch, time, width = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        # Split feature coordinates across heads, retaining token order.
        def split(tensor):
            return tensor.view(batch, time, self.heads,
                               self.head_width).transpose(1, 2)
        q, k, v = map(split, (q, k, v))
        scores = q @ k.transpose(-2, -1) / math.sqrt(self.head_width)
        future = torch.ones(time, time, device=x.device,
                            dtype=torch.bool).triu(1)
        weights = scores.masked_fill(future, float('-inf')).softmax(-1)
        mixed = (weights @ v).transpose(1, 2).contiguous()
        return self.output(mixed.view(batch, time, width))

class Block(nn.Module):
    def __init__(self, width, heads):
        super().__init__()
        self.norm1, self.norm2 = nn.LayerNorm(width), nn.LayerNorm(width)
        self.attention = Attention(width, heads)
        self.feed_forward = nn.Sequential(nn.Linear(width, 4 * width),
                                          nn.GELU(),
                                          nn.Linear(4 * width, width))
```

```
    def forward(self, x):
        x = x + self.attention(self.norm1(x))
        return x + self.feed_forward(self.norm2(x))

class Decoder(nn.Module):
    def __init__(self, vocabulary, width, heads, layers, context):
        super().__init__()
        self.context = context
        self.tokens = nn.Embedding(vocabulary, width)
        self.positions = nn.Embedding(context, width)
        self.blocks = nn.Sequential(*[Block(width, heads)
                                      for _ in range(layers)])
        self.norm = nn.LayerNorm(width)
        self.to_logits = nn.Linear(width, vocabulary)

    def forward(self, ids):
        length = ids.shape[1]
        positions = torch.arange(length, device=ids.device)
        x = self.tokens(ids) + self.positions(positions)
        return self.to_logits(self.norm(self.blocks(x)))

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--width', type=int, default=32)
    parser.add_argument('--heads', type=int, default=4)
    parser.add_argument('--layers', type=int, default=1)
    parser.add_argument('--context', type=int, default=32)
    parser.add_argument('--steps', type=int, default=40)
    parser.add_argument('--lr', type=float, default=0.003)
    parser.add_argument('--seed', type=int, default=7)
    args = parser.parse_args()
    if (min(args.width, args.heads, args.layers, args.context, args.steps) < 1
            or args.width % args.heads or args.lr <= 0):
        parser.error('Use positive values; width must divide evenly by heads.')
    torch.set_num_threads(1)
    torch.manual_seed(args.seed)
    text = ('remind me to call maya. save a note. ask for a time. ' * 40)
    vocabulary = sorted(set(text))
    encode = {char: i for i, char in enumerate(vocabulary)}
    data = torch.tensor([encode[char] for char in text])
    if args.context >= len(data) - 1:
        parser.error('Context is too long for the toy corpus.')
    model = Decoder(len(vocabulary), args.width, args.heads,
                    args.layers, args.context)
    optimizer = torch.optim.AdamW(model.parameters(), lr=args.lr)
    print('Trainable parameters:', sum(p.numel() for p in model.parameters()))
    # Repeated synthetic data is for mechanics, not independent evaluation.
    for step in range(args.steps):
        starts = torch.randint(len(data) - args.context - 1, (8,))
        x = torch.stack([data[i:i+args.context] for i in starts])
        y = torch.stack([data[i+1:i+args.context+1] for i in starts])
        logits = model(x)
        loss = F.cross_entropy(logits.reshape(-1, len(vocabulary)),
                               y.reshape(-1))
        optimizer.zero_grad(set_to_none=True)
        loss.backward()
        optimizer.step()
        if step == 0 or (step + 1) % 10 == 0:
            print('Step', step + 1, 'training loss', round(loss.item(), 4))
    model.eval()
    with torch.no_grad():
        # Change the final input; earlier causal outputs must stay identical.
        probe = data[:args.context].unsqueeze(0).clone()
        altered = probe.clone()
        altered[0, -1] = (altered[0, -1] + 1) % len(vocabulary)
```

```
        torch.testing.assert_close(model(probe)[:, :-1],
                                   model(altered)[:, :-1])
        assert torch.isfinite(loss)
        ids = torch.tensor([[encode['r']]])
        for _ in range(60):
            logits = model(ids[:, -args.context:])[:, -1]
            next_id = logits.argmax(-1, keepdim=True)
            ids = torch.cat([ids, next_id], dim=1)
        print('Toy greedy sample:', ''.join(vocabulary[i] for i in ids[0]))
    print('Causal check passed. Training loss is not a quality benchmark.')

if __name__ == '__main__':
    main()
```

The program prints parameter count, periodic training loss and a greedy sample. The corpus repeats a few sentences, so a decreasing loss mainly establishes that the training machinery can fit that narrow distribution. Independent data and broader tasks are needed for claims about generalization.

Check your understanding: test the mask, not just the loss

The built-in test changes the last input ID and asserts that earlier-position outputs remain the same. Those outputs must not depend on a future token. This verifies a causal property that a low training loss cannot establish.

For a controlled experiment, change only training steps, learning rate or width and retrain from scratch. Record the starting configuration and loss trajectory, not just the final generated sentence. When changing width, keep head divisibility valid. When deliberately removing the mask to learn from a failure, use a disposable copy and restore it afterward.

Explain the complete mechanism

Question: What does this model actually learn, and how does it predict?

Answer: We supply the tokenizer, architecture and loss. Training adjusts embedding and layer parameters to increase probability on the corpus's correct next characters. Prediction looks up input vectors, mixes visible context through attention and nonlinear layers, and produces vocabulary scores. Generation selects one token from those scores and repeats. That explanation connects every earlier lesson without treating the code as a mysterious formula.